Online on xv6 System Call

Section: C1 — **Time:** 30 minutes

Our favourite os xv6 lets a process call any system call as many times as it wants. In this online, you will add a mechanism that lets a process put a cap on how many times it is allowed to use a particular system call.

Required System Call:

1. `limitsyscall(int syscall_num, int limit)` : the calling process may make at most limit more calls to the system call numbered syscall_num. Every call after that must fail immediately and return -1, without the system call actually being carried out. Returns 0 on success and -1 if the arguments are invalid.

The limit must apply only to the process that called limitsyscall. Other processes, including the ones it creates with fork(), must **not** be affected. A process that never calls limitsyscall is **not** limited in any way, and a newly created process must **never** inherit a limit left behind by a process that has already exited.

Required User Program:

1. `limit [syscall_num limit]` , which sets the limit if it is given one, then calls getpid() five times printing each result, forks, and has the child do the same.

The body of the program is given below.

```
// Call getpid() five times, printing what each call returned.
static void burst(char *who)
{
    int i;

    for(i = 1; i <= 5; i++)
        printf("%s %d: getpid() = %d\n", who, i, getpid());
}

int main(int argc, char *argv[])
{
    if(argc == 3){
        if(limitsyscall(atoi(argv[1]), atoi(argv[2])) < 0){
            fprintf(2, "%s: limitsyscall failed\n", argv[0]);
            exit(1);
        }
    } else if(argc != 1){
        fprintf(2, "Usage: %s [syscall_num limit]\n", argv[0]);
        exit(1);
    }

    burst("parent");

    if(fork() == 0){          // the child must not inherit the limit
        burst("child ");
        exit(0);
    }
    wait(0);

    exit(0);
}
```

See the sample I/O for clarification:

```
$ limit 11 3
parent 1: getpid() = 3
parent 2: getpid() = 3
parent 3: getpid() = 3
parent 4: getpid() = -1
parent 5: getpid() = -1
child  1: getpid() = 4
child  2: getpid() = 4
child  3: getpid() = 4
child  4: getpid() = 4
child  5: getpid() = 4
$ limit 5 3
parent 1: getpid() = 5
parent 2: getpid() = 5
parent 3: getpid() = 5
parent 4: getpid() = 5
parent 5: getpid() = 5
child  1: getpid() = 6
child  2: getpid() = 6
child  3: getpid() = 6
child  4: getpid() = 6
child  5: getpid() = 6
$ limit
parent 1: getpid() = 7
parent 2: getpid() = 7
parent 3: getpid() = 7
parent 4: getpid() = 7
parent 5: getpid() = 7
child  1: getpid() = 8
child  2: getpid() = 8
child  3: getpid() = 8
child  4: getpid() = 8
child  5: getpid() = 8
```

Explanation:

- 11 is the system call number of `getpid` and 5 is that of `read`.
- In the first run the parent is allowed three calls to `getpid`; the fourth and fifth are refused and return -1. The child it forks is not limited at all, so all five of its calls succeed.
- The second run limits `read` instead, so `getpid` is untouched in both processes and every call succeeds.
- The third run is given no arguments, so `limitsyscall` is never called and nothing is limited. This process reuses the slot of a process that had a limit and has since exited, so it must still be completely unrestricted.
- Each command is a new process, which is why the pids differ from one run to the next. The pids on your machine may differ; the pattern of values must not.

Submission:

```
git add --all
git diff HEAD > ../{studentID}.patch
```